ZIM School - Use Cases

06. Grading: grade sheets, opening and locking periods, change requests and card slips

Modules: Grading
11 use cases (changes) · 10 views and reports
Written from the use case commands, handlers, policies and rules in `src/` on 2026-11-10.

1. What this area does

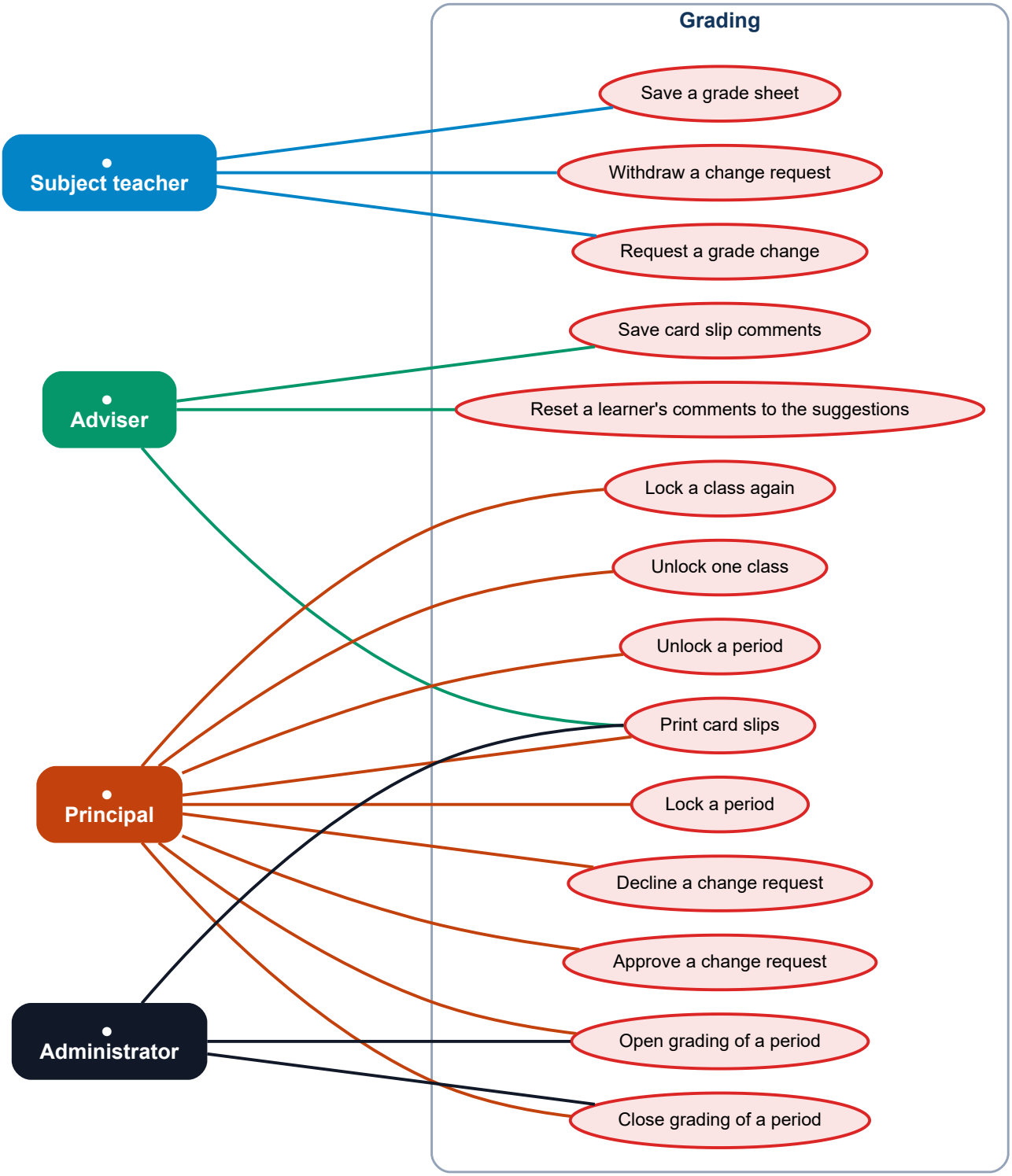
Teachers record each learner's whole-number grade (60-100) per subject and grading period on a **grade sheet**. The school controls when grades may change: the principal or an administrator **opens** and **closes** a period for grading at the school; the principal **locks** it when grades are final, and may **unlock** a single class for corrections. Once a period is locked, a teacher who needs a correction sends a **grade change request**, which the principal approves (grades change at once) or declines.

Only Grade 1-10 sections of the current school year are graded here, and only subjects without components (a parent subject such as MAPEH gets the average of its components). Advisers write the **card slip** comments of their class and print card slips (report cards) with grades, attendance and the descriptor of the general average.

Who may do what (`GradePolicy`): only a load's current teacher enters grades; the adviser, the principal and administrators view; only the principal locks and unlocks; the principal and administrators open and close; only the sender withdraws a request and only the school's principal decides it. **Locks**: a sheet save takes `grades-<section>-<subject>` plus a shared hold on `grade-lock-<school>`; the principal's period changes take `grade-lock-<school>` exclusively, so no sheet can be saved while a period is being locked.

2. Use case map

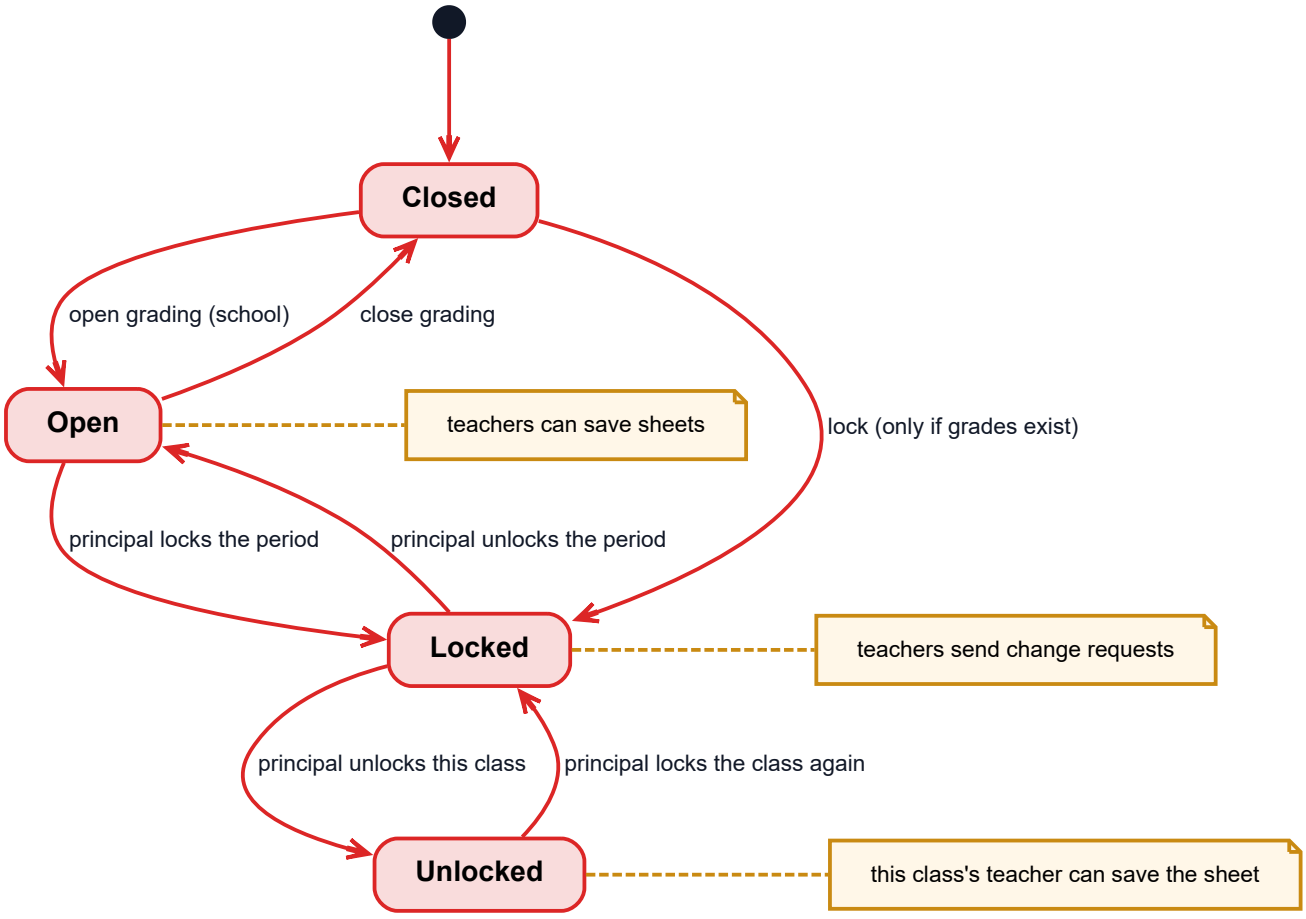
A standard use case diagram: the actors on the left, each use case as an ellipse inside the system frame, and a line for each use case an actor starts. A dashed line has a note on when it applies.



Subject teacher · Adviser · Principal · Administrator — Line colour = the actor who starts it.

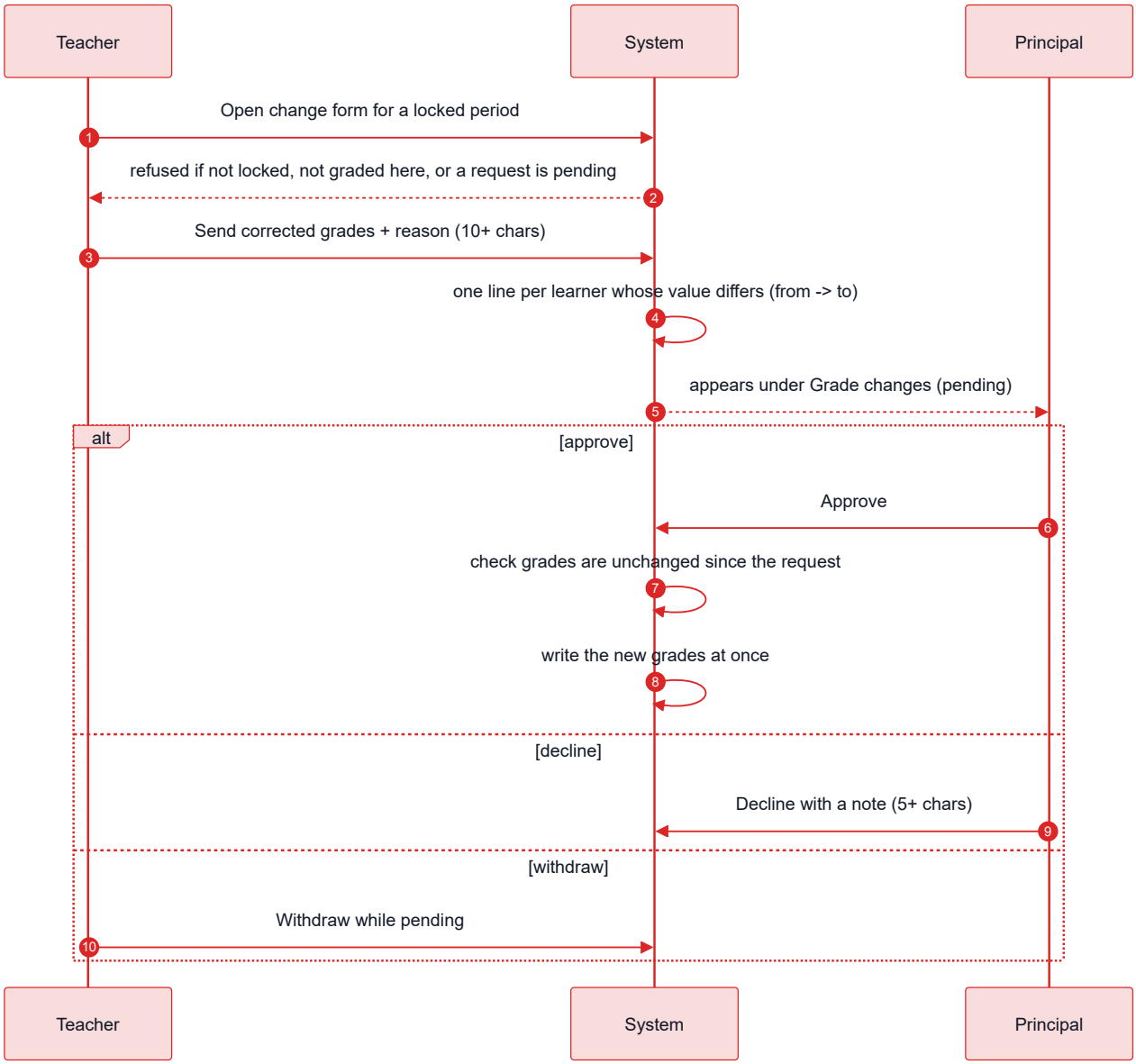
Grading — Ellipse colour = the module the use case belongs to.

3.1 Flow: Status of a class's grades for a period



Grading Drawn in the colour of the module this flow belongs to.

3.2 Flow: Grade change request



Grading Drawn in the colour of the module this flow belongs to.

4. Use cases (changes)

Each card is one use case command sent through the use case bus. It runs in one database transaction: if any rule refuses, nothing is saved and the message in *Alternatives and refusals* is shown to the user.

GRD-01 Save a grade sheet

Primary actor	Subject teacher
Goal	Enter or change the grades of one class (section + subject) for a grading period.
Where (route)	`teacher_grades_sheet /teacher/grades/{load}/{period}`
Preconditions	• The teacher is the load's current teacher; the class is gradable; the period is Open, or the class is Unlocked.
Main flow	1. The teacher opens the sheet from My grades; it lists the learners who count in the section. 2. Types grades (blank = no grade yet) and saves. 3. The system takes the class lock and a shared hold on the school's grade lock, re-reads the grades and saves only changed values. 4. "Saved <period> grades of <class>" is logged.
Alternatives and refusals	• "You cannot enter grades for this class." • "<period> isn't open for grading." • "<period> is locked by the principal." • "Enter a grade from 60 to 100." • Busy: "Another change is being saved. Please try again."
Result (data changed)	Grade rows created or changed (updated_by, updated_at).
In the code	`SaveGradeSheetCommand → GradeRecorder::saveSheet`

GRD-02 Open or close grading of a period

Primary actor	Principal, Administrator
Goal	Let the school's teachers enter grades for a period, or stop them.
Where (route)	`school_grades_open / school_grades_close (/schools/{id}/grades/{period}/...)`
Preconditions	• Manages the school; the period belongs to the current school year; it is not locked.
Main flow	1. On the school's Grades page the principal presses Open (or Close) for the period. 2. The system takes the school's grade lock and adds (or removes) the GradeOpening; an admin's log line names the school.
Alternatives and refusals	• "<period> is already open." / "<period> is already closed." • "<period> is locked. Unlock it first."
Result (data changed)	GradeOpening created or removed.
In the code	`OpenGradingPeriodCommand, CloseGradingPeriodCommand`

GRD-03 Lock a period

Primary actor	Principal
Goal	Make a period's grades final for the whole school.
Where (route)	`school_grades_lock /schools/{id}/grades/{period}/lock`
Preconditions	• Principal of the school; the period is open, or the school already has grades in it.
Main flow	1. The principal reviews the progress of every class and confirms Lock. 2. The system removes any class unlocks, adds the GradeLock and logs "Locked <period> grades".
Alternatives and refusals	• "<period> is already locked." • "<period> hasn't been opened for grading yet."
Result (data changed)	GradeLock created; GradeUnlock rows of the period removed.
In the code	`LockGradingPeriodCommand`

GRD-04 Unlock a period

Primary actor	Principal
Goal	Undo a lock (e.g. locked too early).
Where (route)	`school_grades_unlock /schools/{id}/grades/{period}/unlock`
Preconditions	• The period is locked.
Main flow	1. The principal presses Unlock; the GradeLock is removed. The period is Open or Closed again depending on its opening.
Alternatives and refusals	• "<period> is not locked."
Result (data changed)	GradeLock removed.
In the code	`UnlockGradingPeriodCommand`

GRD-05 Unlock one class, or lock it again

Primary actor	Principal
Goal	Let one class's teacher fix grades in a locked period without unlocking the whole school.
Where (route)	`school_grades_unlock_class / school_grades_lock_class`
Preconditions	• The period is locked; the class is gradable.
Main flow	1. On a section's grades the principal unlocks the subject; the teacher can now save that sheet. 2. When done, the principal locks the class again.
Alternatives and refusals	• "<period> is not locked." • "This class is already unlocked." • "This class is not unlocked."
Result (data changed)	GradeUnlock created or removed.
In the code	`UnlockClassGradesCommand, LockClassGradesCommand`

GRD-06 Request a grade change

Primary actor	Subject teacher
Goal	Ask the principal to correct grades of a locked period.
Where (route)	`teacher_grades_change /teacher/grades/{load}/{period}/change`
Preconditions	• The teacher's class; the period is locked (and the class not unlocked); no pending request for the class.
Main flow	1. The teacher opens the change form, enters the corrected grades and a reason. 2. The system keeps one line per learner whose value differs (from, to) and saves the request as pending.
Alternatives and refusals	• "This class isn't graded here." • "<period> isn't locked - change the grades on the sheet." • "You already have a pending change request for this class." • "Write the reason for the change (at least 10 characters)." • "Enter a grade from 60 to 100." • "Enter at least one corrected grade."
Result (data changed)	GradeChangeRequest (pending) + GradeChangeLine rows.
In the code	`RequestGradeChangeCommand → GradeChangeManager::request`

GRD-07 Approve a grade change

Primary actor	Principal
Goal	Accept the correction; the grades change at once.
Where (route)	`school_grade_change_approve /schools/{id}/grade-changes/{request}/approve`
Preconditions	• Principal of the school; the request is pending.
Main flow	1. The principal opens Grade changes, reviews from/to values and the reason, and approves. 2. The system takes the class lock and the school's grade lock, checks the grades have not changed since the request, writes the new values and marks it approved.
Alternatives and refusals	• "Grades have changed since this request was sent. Decline it so the teacher can send a new one." • "This request has already been decided."
Result (data changed)	Grade values; GradeChangeRequest.status = approved, decided_by, decided_at.
In the code	`ApproveGradeChangeCommand`

GRD-08 Decline a grade change

Primary actor	Principal
Goal	Refuse the correction with a note for the teacher.
Where (route)	`school_grade_change_decline /schools/{id}/grade-changes/{request}/decline`
Preconditions	• The request is pending.
Main flow	1. The principal writes a note and declines.
Alternatives and refusals	• "Write a note for the teacher (at least 5 characters)." • "This request has already been decided."
Result (data changed)	GradeChangeRequest.status = declined, decision_note.
In the code	`DeclineGradeChangeCommand`

GRD-09 Withdraw a grade change request

Primary actor	Subject teacher (the sender)
Goal	Take back a request that is still waiting.
Where (route)	`teacher_grades_change_withdraw /teacher/grades/changes/{request}/withdraw`
Preconditions	• The teacher sent it; it is pending.
Main flow	1. The teacher presses Withdraw on the sheet.
Alternatives and refusals	• "This request has already been decided."
Result (data changed)	GradeChangeRequest.status = withdrawn.
In the code	`WithdrawGradeChangeCommand`

GRD-10 Save card slip comments

Primary actor	Adviser
Goal	Choose the comments printed on each learner's card slip for a term.
Where (route)	`teacher_class_card_slips /teacher/class/card-slips`
Preconditions	• The teacher advises a section this school year.
Main flow	1. The adviser opens Card slips for a term; each learner has suggested comments (from grades and attendance: excellent / very good / good performance, improvement, needs to improve in <subjects>, incomplete grades, perfect attendance, frequent absences, often late ...). 2. The adviser ticks built-in comments and may type one own comment per learner, then saves. 3. A learner left on the suggestions with no typed comment is not stored (a saved choice is removed), so their slip keeps following the suggestions as the grades change.
Alternatives and refusals	• "No such grading period." • Not the adviser (403).
Result (data changed)	CardSlipComment rows (codes JSON + custom text).
In the code	`SaveCardSlipCommentsCommand → CardSlipCommentRecorder::save`

GRD-11 Reset a learner's card slip comments

Primary actor	Adviser
Goal	Go back to the suggested comments for one learner.
Where (route)	`teacher_class_card_slip_reset /teacher/class/card-slips/{period}/{enrollment}/reset`
Preconditions	• The learner is in the adviser's class.
Main flow	1. The adviser presses Reset; the stored choice is removed.
Alternatives and refusals	• "No such learner in your class."
Result (data changed)	CardSlipComment removed.
In the code	`ResetCardSlipCommentCommand`

5. Views, downloads and reports (read only)

Each row is one query sent through the query bus. Queries change nothing.

Page / file	Who	Route	What it shows	Query
My grades	Subject teacher	`teacher_grades` `/teacher/grades`	Every class the teacher grades with each period's progress and status.	`GetTeacherGradesCommand`
Grade sheet	Teacher, Adviser, Principal	`teacher_grades_sheet`	A class's sheet, its status, change requests and the section grid.	`GetGradeSheetCommand`
Change form	Subject teacher	`teacher_grades_change`	The form, or why a request can't be sent now.	`GetGradeChangeFormCommand`
Class grades	Adviser	`teacher_class_grades` `/teacher/class/grades`	All subjects of the advised section for a period.	`GetClassGradesCommand`
School grades overview	Principal, Administrator	`school_grades` `/schools/{id}/grades` `(principal_grades)`	Every class's progress for a period, opening/lock state, pending requests.	`GetSchoolGradesCommand`, `GetGradeLockCommand`
A section's grades	Principal, Administrator	`school_grades_section`	Every subject of a section for a period.	`GetSectionGradesCommand`
Failing learners	Principal, Administrator	`school_grades_failing`	Learners who failed a period's general average.	`GetFailingLearnersCommand`
Grade change requests	Principal	`school_grade_changes` `/schools/{id}/grade-changes`	Pending or decided requests this year.	`GetGradeChangesCommand`, `GetGradeChangeCommand`

Card slips page	Adviser	`teacher_class_card_slips`	Each learner's slip preview and comments for a term.	`GetCardSlipsCommand`
Card slip PDF	Adviser, Principal, Administrator	`teacher_class_card_slips_pdf`, `teacher_class_card_slip_pdf`, `school_section_card_slips_pdf`	Report cards: grades, general average and DepEd descriptor, attendance (from the Attendance API), comments.	`GetCardSlipPdfCommand`